U.S. Chronic Disease Landscape:

A Data Science Perspective

GROUP MEMBERS

- ☐ Dr. Pala Jijol Onesimos – Resident Doctor (MD.FCCM.)
- ☐ Shreya Shrestha – Public Health Practitioner
- ☐ Dhwani Mehta – Biotechnologist
- ☐ Sahil Patel - Chemist
- ☐ Sohil Chaudhary - Registered Nurse

DESCRIPTIVE TITLE

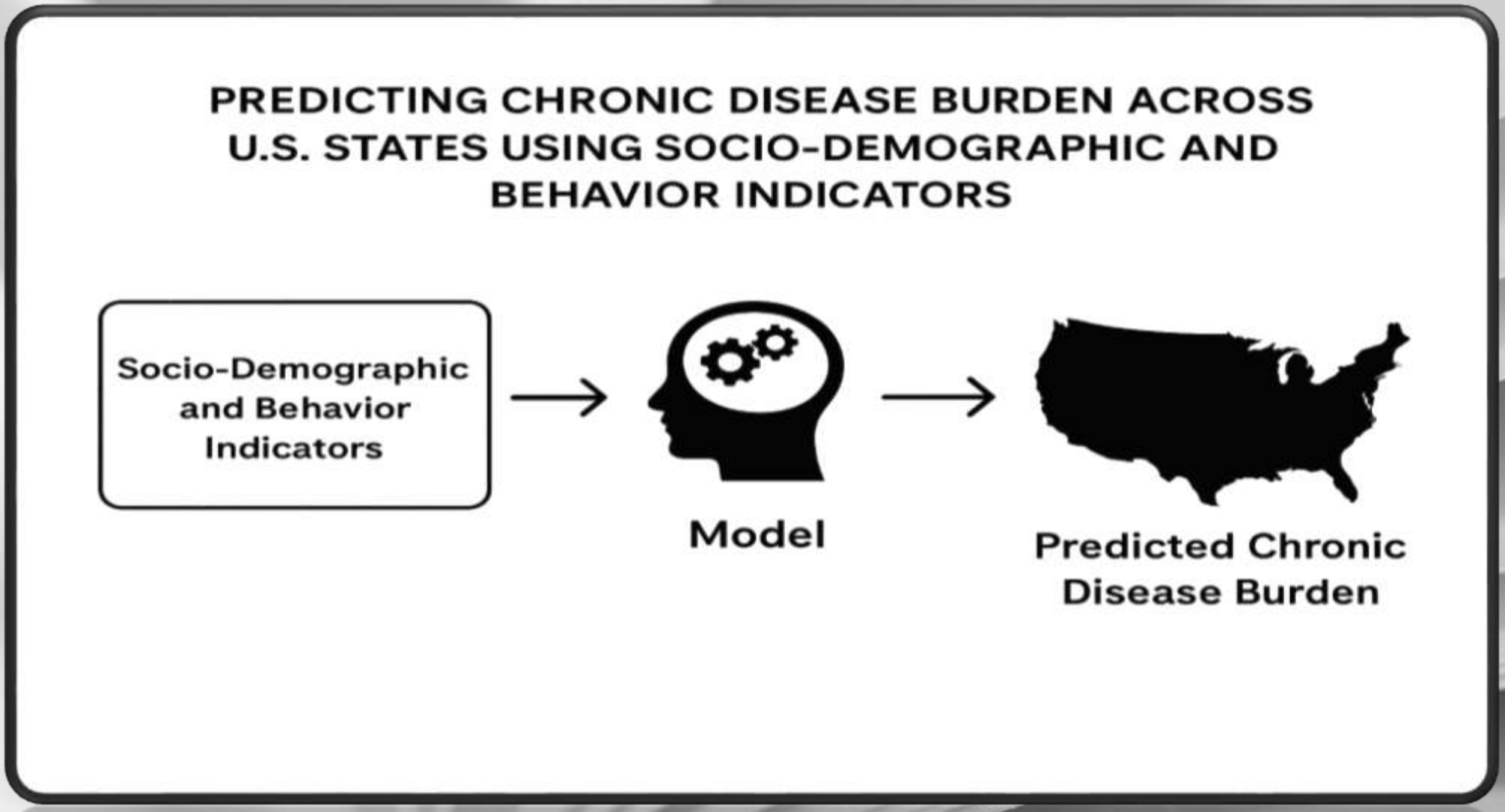


TABLE OF CONTENTS :

DATA DESCRIPTION

Dataset: U.S. Chronic Disease Indicators (CDI)

Source: CDC via Data.gov

Format: CSV (downloadable via API)

Contents: 115 indicators covering chronic diseases and associated risk factors, stratified by state, year, demographics.

Use Case: Enables predictive modeling of disease burden (e.g., diabetes, heart disease) across U.S. geographies.

DATA DESCRIPTION CONTINUED

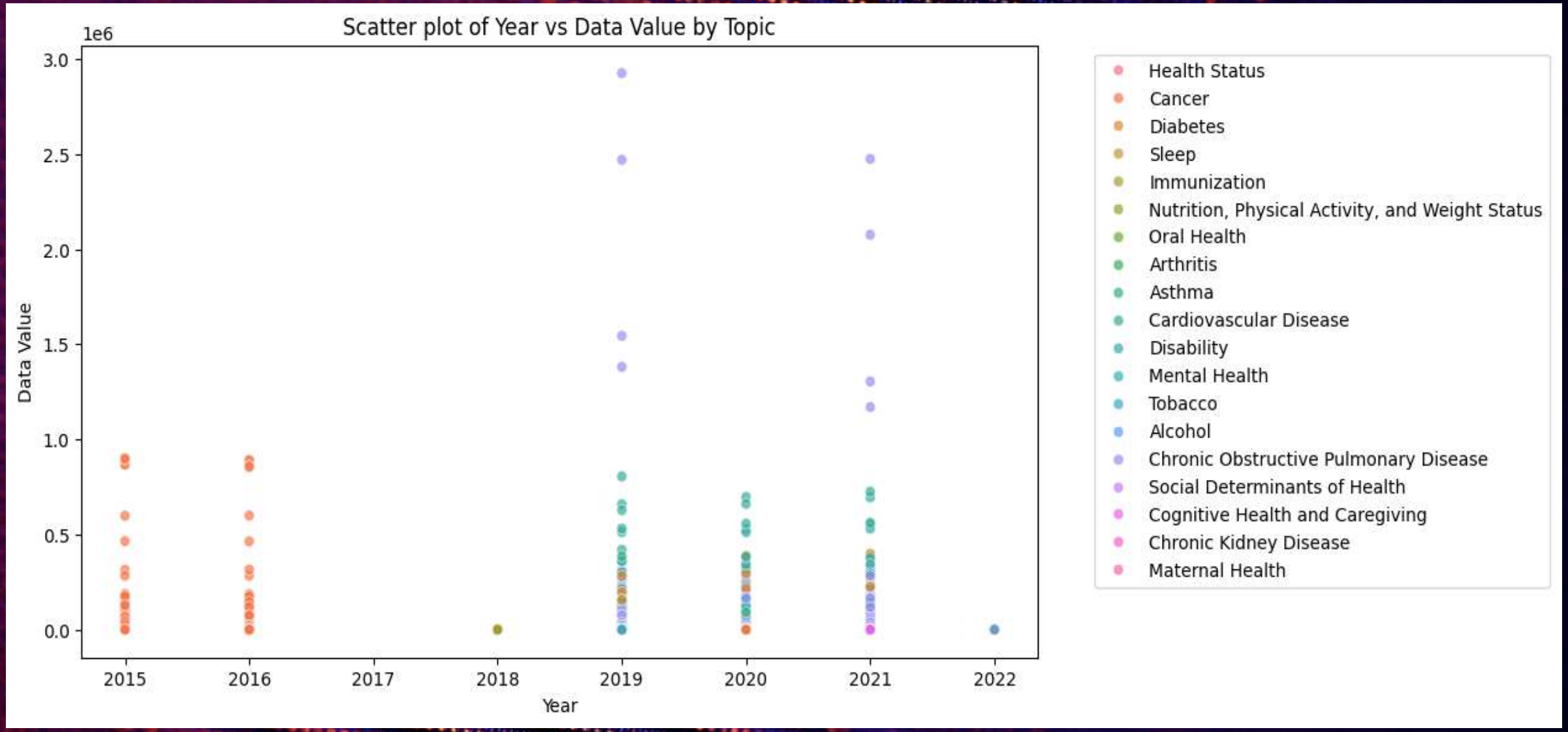
The dataset provides 115 indicators developed by consensus among the CDC, the Council of State and Territorial Epidemiologists, and the National Association of Chronic Disease Directors. These indicators allow states and territories to uniformly define, collect, and report chronic disease data that are important to public health practice in their area.

COLUMN NAME & DATA DESCRIPTION

Column Name	Data Description
LocationAbbr	Abbreviation of the state or territory (e.g., 'CA' for California).
LocationDesc	Full name of the state or territory (e.g., 'California').
YearStart	Starting year of the data reporting period.
YearEnd	Ending year of the data reporting period.
Topic	The health topic category (e.g., 'Diabetes', 'Cancer').

COLUMN NAME & DATA DESCRIPTION

Column Name	Data Description
Indicator	Specific health indicator (e.g., 'Percentage of adults with diabetes')
StratificationCategoryID1/2/3	Demographic breakdowns (age, gender, race, etc)
DataValueType	Type of data value (e.g., 'Crude Prevalence', 'Age-Adjusted Rate').
DataValueUnit	The unit of measurement (e.g. % rate per 1000,000 population)
DataValue	The numeric measure of indicator (main outcome value)

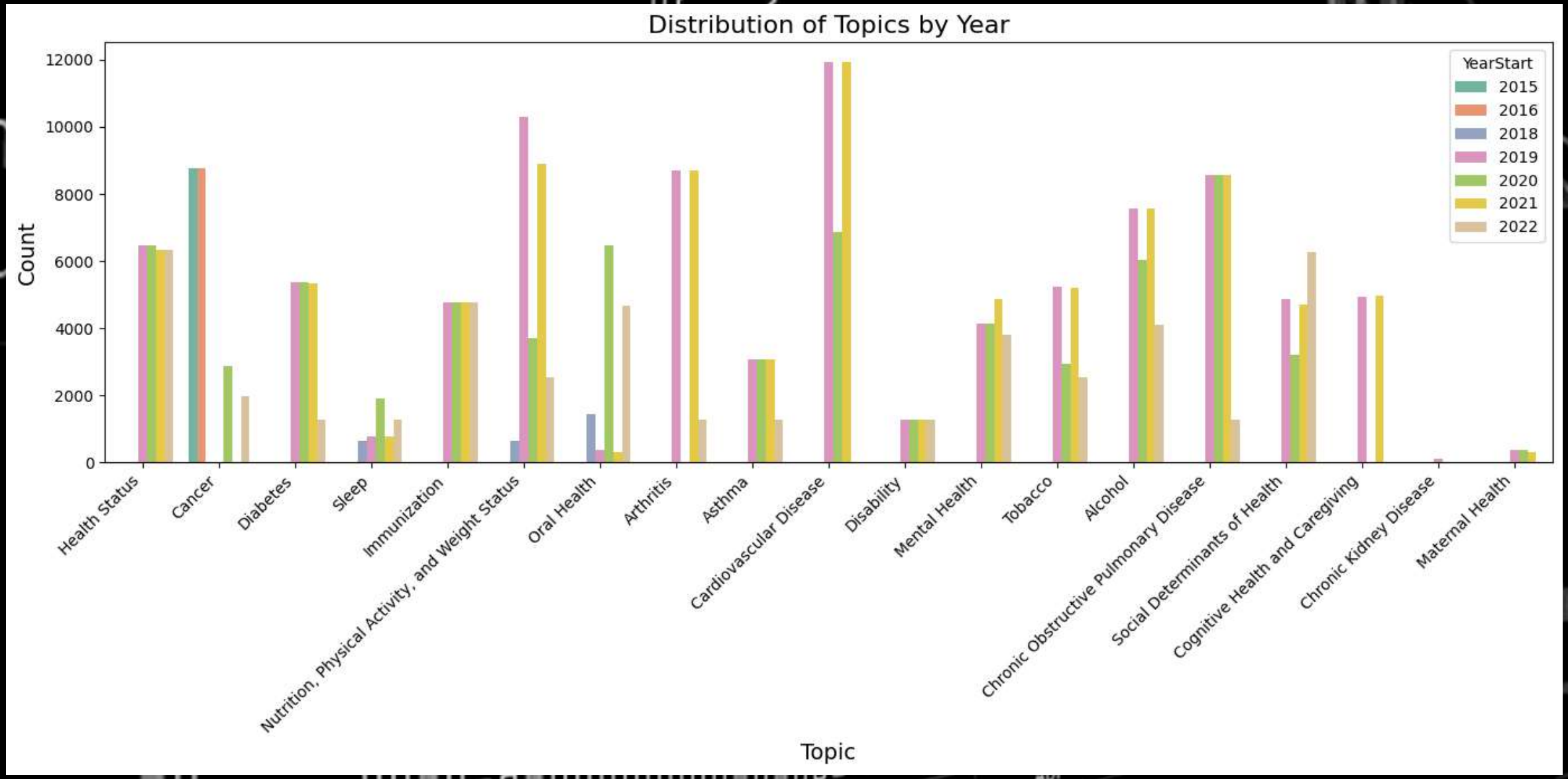

Scatter plot of Year vs Data Value by Topic
1e6
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Data Value
2015
2016
2017
2018
2019
2020
2021
2022
Year
Health Status
Cancer
Diabetes
Sleep
Immunization
Nutrition, Physical Activity, and Weight Status
Oral Health
Arthritis
Asthma
Cardiovascular Disease
Disability
Mental Health
Tobacco
Alcohol
Chronic Obstructive Pulmonary Disease
Social Determinants of Health
Cognitive Health and Caregiving
Chronic Kidney Disease
Maternal Health

OBSERVATION:

The scatter plot reveals a sharp increase in data values around 2019, especially for chronic conditions like COPD and cancer, with values surpassing 2 million. Earlier years (2015–2016) show smaller, more uniform clusters. High data values persist into 2020–2021, while preventive topics such as immunization remain consistently low.

INTERFACE:

The spike in 2019 may reflect intensified health monitoring efforts, possibly due to pre-pandemic initiatives or enhanced reporting systems. Continued high levels during 2020–2021 could be linked to the impact of COVID-19, which likely heightened attention on respiratory and chronic conditions. The consistently low tracking of preventive areas points to a healthcare system that emphasizes disease management over prevention, which may have long-term cost and health equity implications.

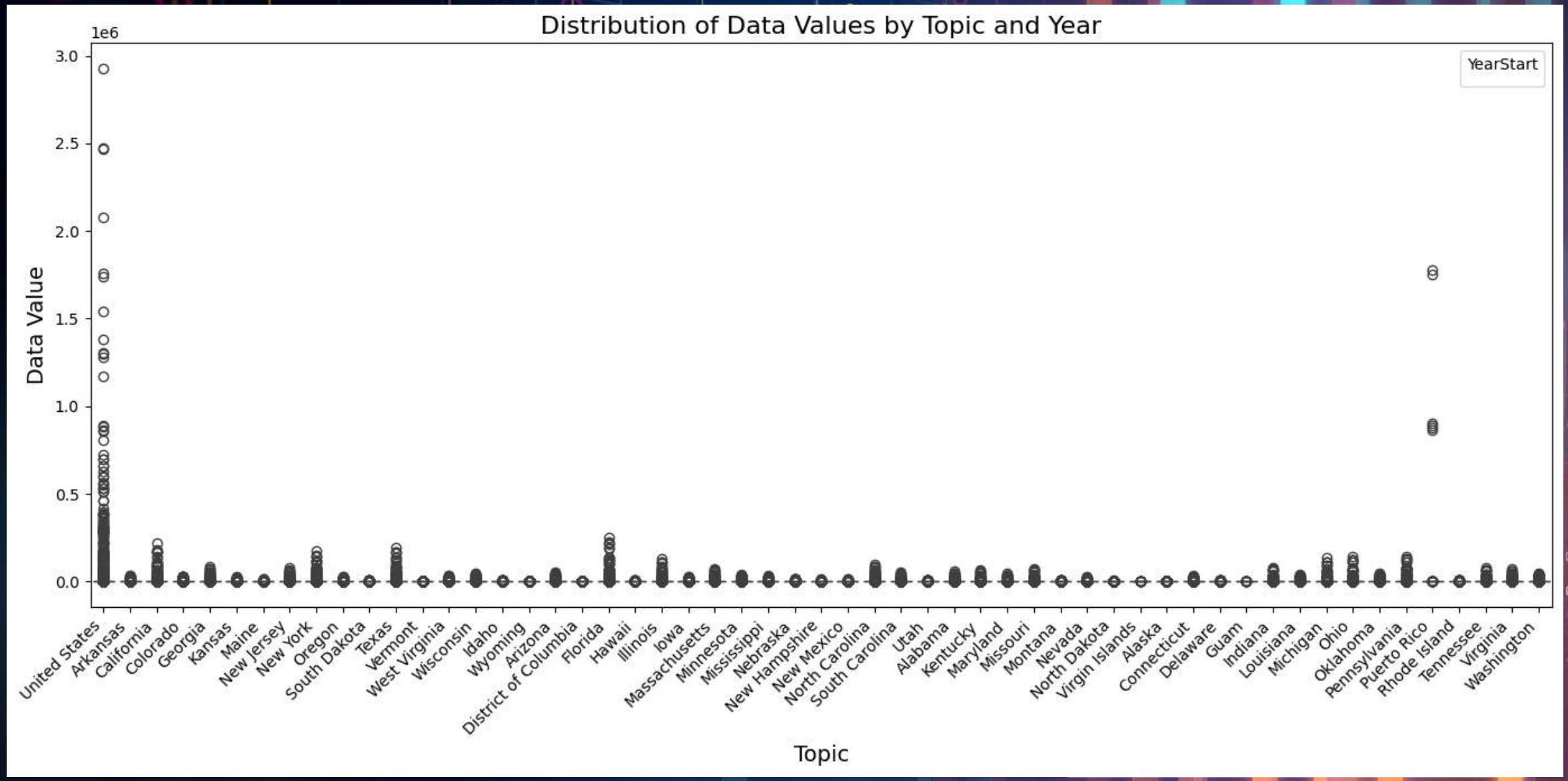

Distribution of Topics by Year
Count
Topic
YearStart
2015
2016
2018
2019
2020
2021
2022
0
2000
4000
6000
8000
10000
12000
Health Status
Cancer
Diabetes
Sleep
Immunization
Nutrition, Physical Activity, and Weight Status
Oral Health
Arthritis
Asthma
Cardiovascular Disease
Disability
Mental Health
Tobacco
Alcohol
Chronic Obstructive Pulmonary Disease
Social Determinants of Health
Cognitive Health and Caregiving
Chronic Kidney Disease
Maternal Health

OBSERVATION:

The bar chart shows how different health topics were represented between 2015 and 2022. Topics such as cardiovascular disease, arthritis and oral health had consistently high counts, while areas like maternal health and sleep were minimally represented. Cancer and health status peaked in earlier years but declined afterward.

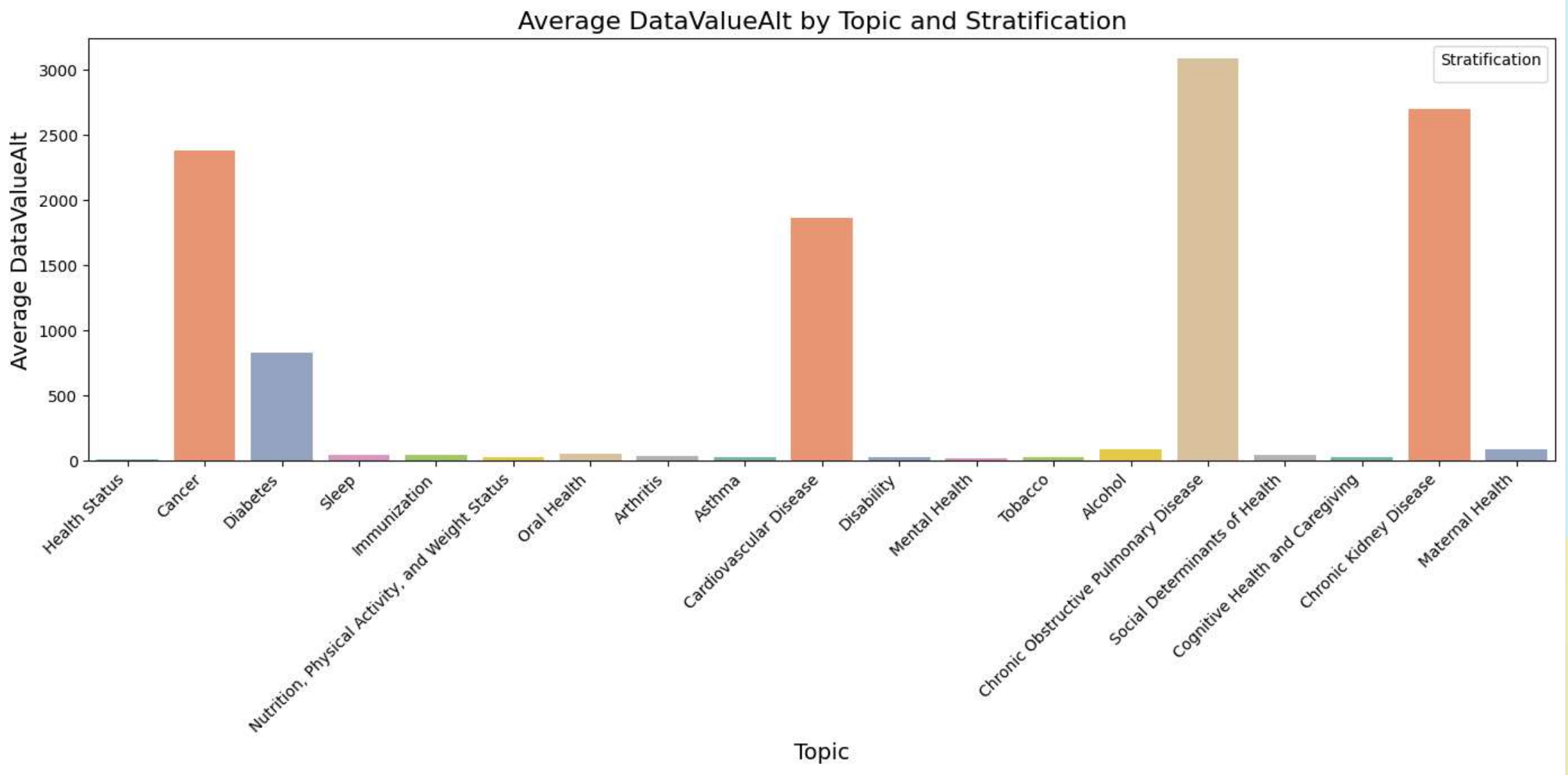
INTERPRETATION:

This pattern suggests that chronic diseases receive sustained public health attention, reflecting their long-term burden on the healthcare system. Preventive health topics, such as nutrition and sleep, appear underrepresented, which could indicate a gap in policy emphasis or research funding. This imbalance may influence future healthcare strategies by prioritizing treatment over prevention.


Distribution of Data Values by Topic and Year
1e6
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Data Value
YearStart
United States
Arkansas
California
Colorado
Georgia
Kansas
Maine
New Jersey
New York
Oregon
South Dakota
Texas
Vermont
West Virginia
Wisconsin
Idaho
Wyoming
Arizona
District of Columbia
Florida
Hawaii
Illinois
Iowa
Massachusetts
Minnesota
Mississippi
Nebraska
New Hampshire
New Mexico
North Carolina
South Carolina
Utah
Alabama
Kentucky
Maryland
Missouri
Montana
Nevada
North Dakota
Virgin Islands
Alaska
Connecticut
Delaware
Guam
Indiana
Louisiana
Michigan
Ohio
Oklahoma
Pennsylvania
Puerto Rico
Rhode Island
Tennessee
Virginia
Washington
Topic

OBSERVATION:
Most topics cluster at relatively low data values across states, but some — particularly cancer and chronic obstructive pulmonary disease (COPD) — show extreme outliers, with values reaching above 1 million. The spread of data values varies widely, with certain conditions being tracked far more extensively.

INTERPRETATION:
These disparities highlight unequal surveillance intensity across topics and regions. High outliers likely correspond to conditions with greater disease burden or stronger monitoring systems. Meanwhile, the clustering of lower values indicates limited tracking for some conditions, which may reduce visibility of smaller but still important public health issues.


Average DataValueAlt by Topic and Stratification
Stratification
Average DataValueAlt
3000
2500
2000
1500
1000
500
0
Health Status
Cancer
Diabetes
Sleep
Immunization
Nutrition, Physical Activity, and Weight Status
Oral Health
Arthritis
Asthma
Cardiovascular Disease
Disability
Mental Health
Tobacco
Alcohol
Chronic Obstructive Pulmonary Disease
Social Determinants of Health
Cognitive Health and Caregiving
Chronic Kidney Disease
Maternal Health
Topic

OBSERVATIONS:
The graph shows large variations in the average DataValueAlt across different health topics. Chronic Obstructive Pulmonary Disease, Chronic Kidney Disease, and Cancer have the highest averages compared to all other topics. Cardiovascular Disease and Diabetes follow at moderately high levels, while most other topics remain below 100. This indicates that only a few conditions dominate the dataset while the rest contribute relatively little.

INTERPRETATION:
The results suggest that chronic diseases like COPD, Kidney Disease, Cancer, and Cardiovascular Disease are major health burdens in comparison to other conditions. These findings may highlight areas where healthcare resources and interventions are most needed. Lower averages in topics such as Sleep, Mental Health, and Immunization might reflect differences in measurement or relatively lower reported impact. Overall, the data emphasizes the priority of addressing chronic diseases in public health planning.

REGRESSION & CONFUSION MATRIX

Code snippet

```
# Linear regression
X = df[['LowConfidenceLimit’]]
y = df['DataValueAlt’]
model = LinearRegression().fit(X, y)
print("y = aX + b")

# Confusion matrix
y_bin = (y > y.median()).astype(int)
clf = LogisticRegression().fit(X, y_bin)
y_pred = clf.predict(X)
print(confusion_matrix(y_bin, y_pred))
```

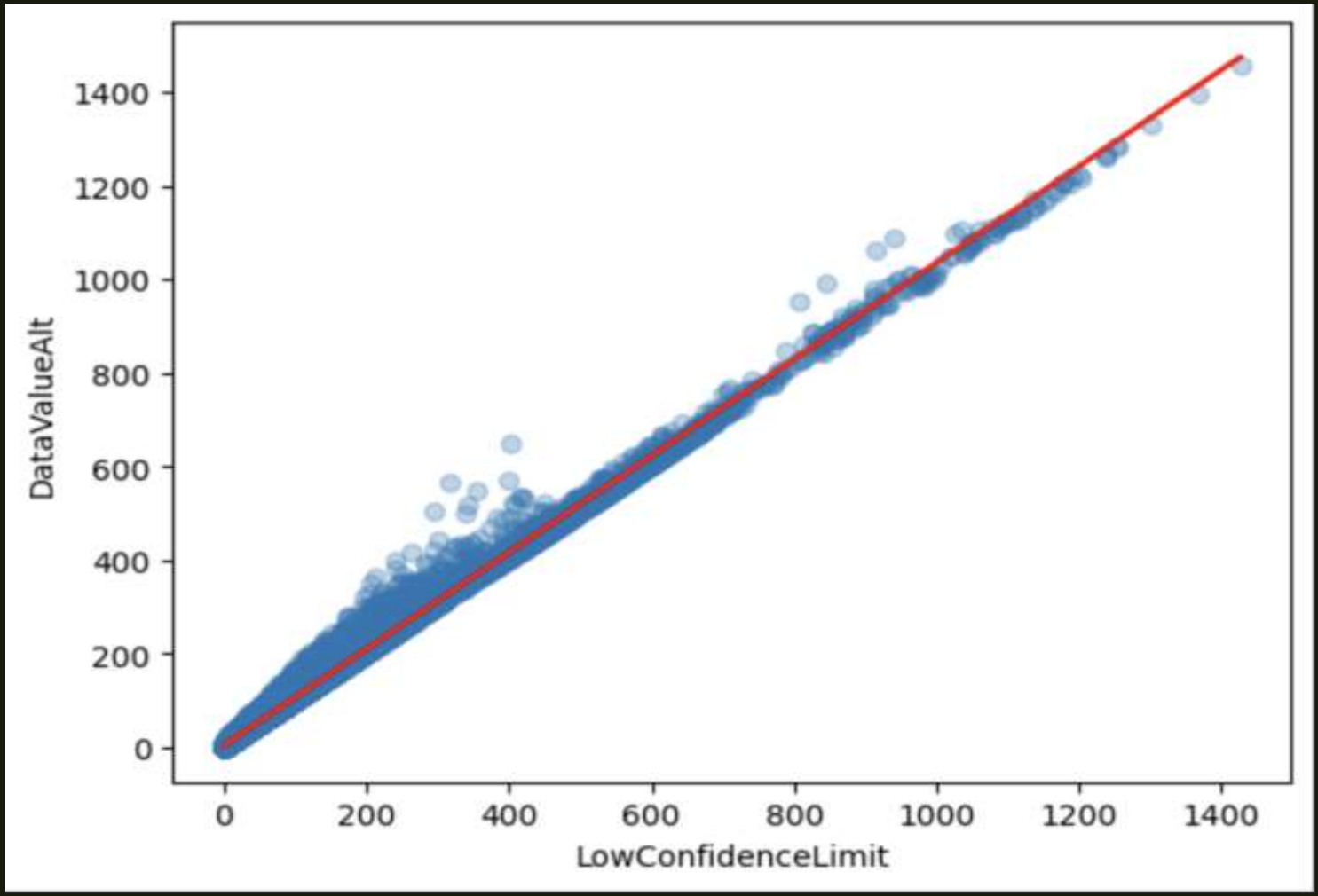
*Formatting may vary in slides, code was executed correctly in Python

Results:

Linear regression : y = 1.03 * X + 3.21

Confusion matrix : [[90557, 3886],
[4650, 89792]]

SCATTER +LINE PLOT



Interpretation

- The slope a shows how much DataValueAlt changes when LowConfidenceLimit
- increases by 1.
- The intercept b is the starting point when
- X =0.

CONFUSION MATRIX

	Predicted Low	Predicted High
Actual Low	TN = 90557	FP = 3886
Actual High	FN = 4650	TP = 89792

Interpretation

The confusion matrix shows how well the logistic model separates “high” vs “low” values.

Accuracy = (TP + TN) / Total → tells how good the good the predictor is for classification.

KNN AND NAIVE BAYES MODEL COMPARISON

Data Preparation (Code Snippet)

```
# Import Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load and Prepare Data
df = pd.read_csv("U.S._Chronic_Disease_Indicators.csv")

data = df[['LowConfidenceLimit', 'HighConfidenceLimit', 'DataValue']].dropna().copy()
median_value = data['DataValue'].median()

# Create Binary Target
data['Target'] = (data['DataValue'] > median_value).astype(int)

# Define Features and Target
X = data[['LowConfidenceLimit', 'HighConfidenceLimit']]
y = data['Target']

# Split Train/Test Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

*Formatting may vary in slides, code was executed correctly in Python

Model Training (Code Snippet)

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score


# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


# Train KNN Model
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)
acc_knn = accuracy_score(y_test, y_pred_knn)


# Train Naive Bayes Model
nb = GaussianNB()
nb.fit(X_train_scaled, y_train)
y_pred_nb = nb.predict(X_test_scaled)
acc_nb = accuracy_score(y_test, y_pred_nb)
```

Confusion Matrices (Code Snippet)

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# Confusion Matrices
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

cm_knn = confusion_matrix(y_test, y_pred_knn)
cm_nb = confusion_matrix(y_test, y_pred_nb)

sns.heatmap(cm_knn, annot=True, fmt="d", cmap="Blues", ax=axes[0])
axes[0].set_title("KNN Confusion Matrix")

sns.heatmap(cm_nb, annot=True, fmt="d", cmap="OrRd", ax=axes[1])
axes[1].set_title("Naive Bayes Confusion Matrix")

plt.tight_layout()
plt.show()
```

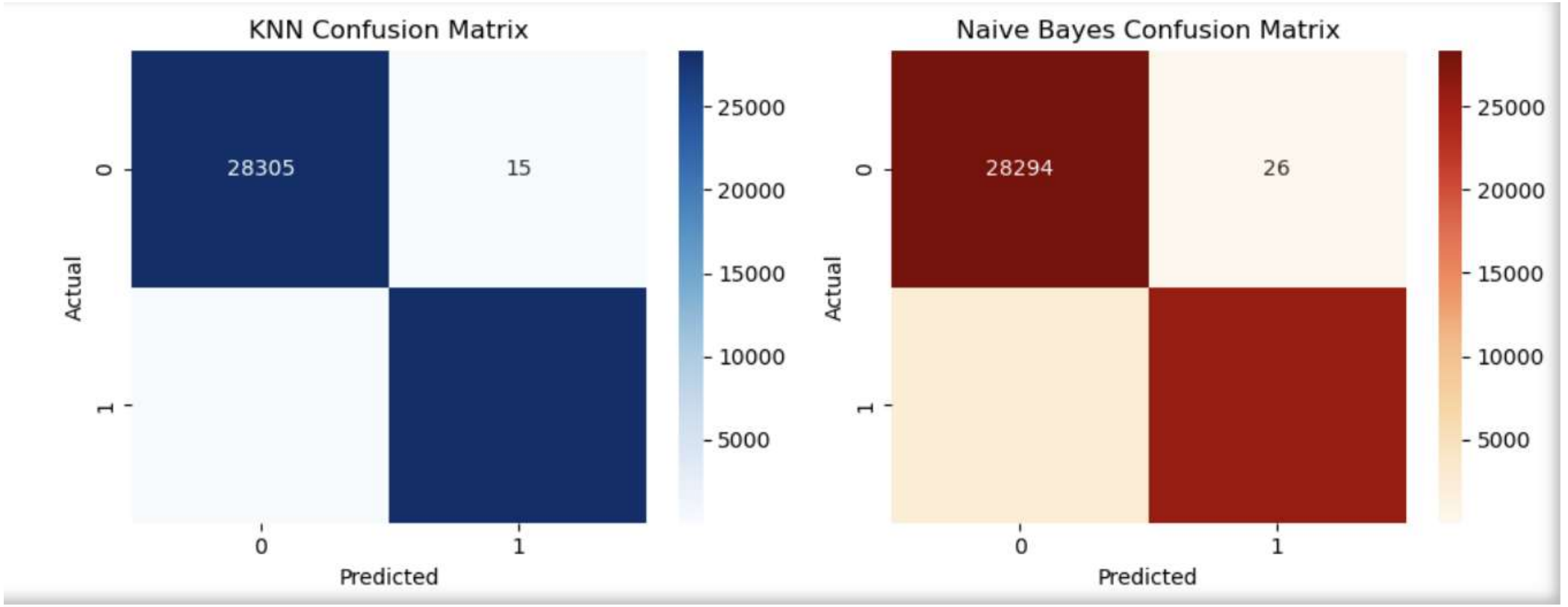
Accuracy Comparison (Code Snippet)

```
# Accuracy Results
print("KNN Accuracy:", round(acc_knn, 3))
print("Naive Bayes Accuracy:", round(acc_nb, 3))

# Accuracy Comparison Graph
models = ['KNN', 'Naive Bayes']
accuracies = [acc_knn, acc_nb]

plt.figure(figsize=(6, 4))
plt.bar(models, accuracies, color=['skyblue', 'salmon'])
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
for i, v in enumerate(accuracies):
    plt.text(i, v + 0.01, f"{v:.2f}", ha='center', fontsize=12)
plt.show()
```

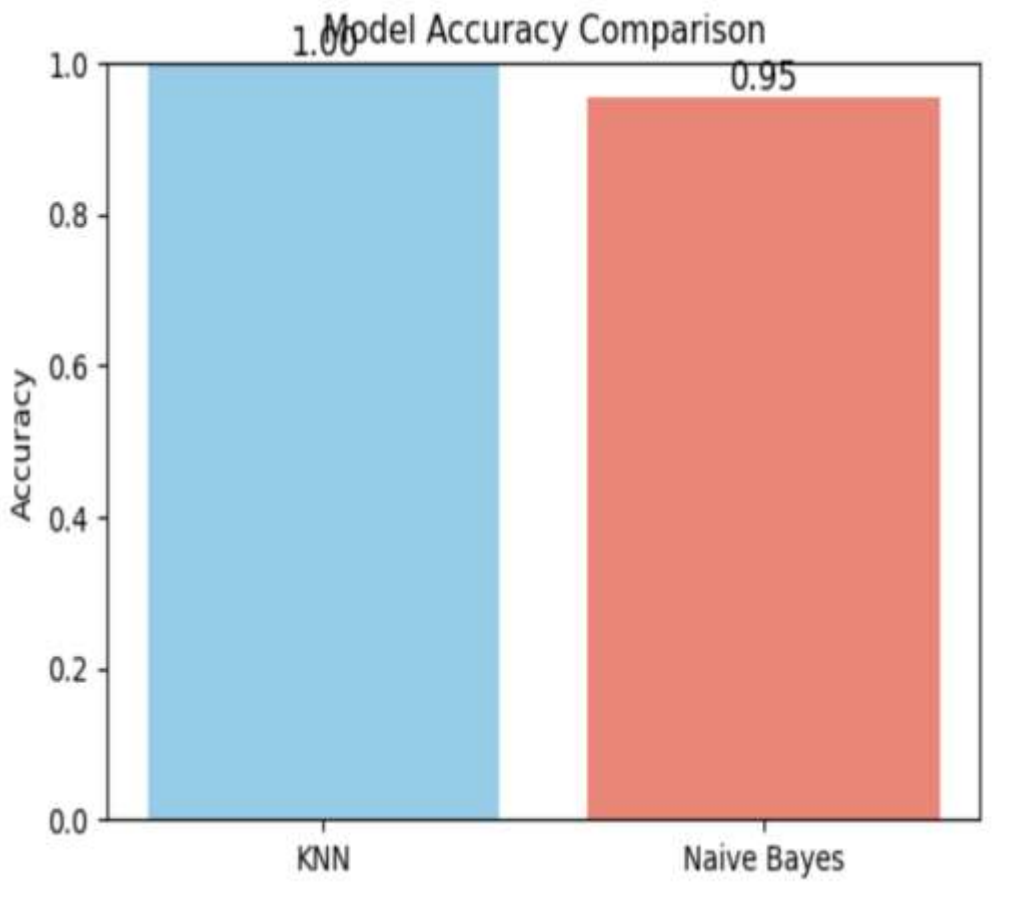
*Formatting may vary in slides, code was executed correctly in Python



The KNN confusion matrix shows more correct predictions along the diagonal, meaning it classified "High" and "Low" values more accurately.

In contrast, Naive Bayes made more misclassifications, confirming that KNN performs better for this dataset.

Model Accuracy Comparision



The **KNN model** achieved higher accuracy (≈0.999) than **Naive Bayes** (≈0.955), showing better prediction performance. This indicates that **KNN works better** for this dataset because it captures relationships between numeric features more effectively.

Model Comparison & Insights

Model	Accuracy	Strenghts	Weakness
KNN	0.999	Captures complex patterns; robust to non-linear data	Sensitive to scaling
Naive Bayes	0.955	Simple, fast, interpretable	Assumes feature independence

KNN produced higher accuracy, likely because it captures small local variations in numeric data.
Naive Bayes assumes features are independent — which might not hold for these correlated limits.

Report and Conclusion

- **KNN Accuracy:** 0.999
- **Naive Bayes Accuracy:** 0.955
- **Better Model:** K-Nearest Neighbors (KNN)
- **Reason:** Performs better on correlated numeric features and captures local variations in data more effectively.

After training and evaluating both models, the K-Nearest Neighbors (KNN) model achieved an accuracy of approximately 0.999, while the Naive Bayes (GaussianNB) model reached about 0.955. This shows that KNN performed slightly better on this dataset. The higher accuracy of KNN suggests it was more effective at capturing the underlying patterns between the numeric features LowConfidenceLimit and HighConfidenceLimit. In contrast, Naive Bayes assumes that features are independent, which may not be true in this dataset since the two confidence limit values are highly correlated. Therefore, KNN works better for this data, as it adapts more effectively to local variations and relationships between features, leading to more accurate classifications of “High” and “Low” DataValue categories.

SUPPORT VECTOR MACHINE (SVM) CLASSIFIERS ON U.S. CHRONIC DISEASE INDICATORS

Data Overview

Dataset: U.S. Chronic Disease Indicators (CDC 2024)

Purpose: Predict whether a state's reported DataValue is above or below the median using SVM classification.

Selected Features:

LowConfidenceLimit – Lower bound of statistical confidence interval for each measure.

HighConfidenceLimit – Upper bound of the confidence interval for each measure.

Target Variable: *DataValue (> median = 1, ≤ median = 0)* → Binary classification label for "High vs Low Indicator Values."

Goal: Train and compare four SVM models (*linear, rbf, polynomial, sigmoid*) to identify the most accurate classifier for these features and interpret their performance.

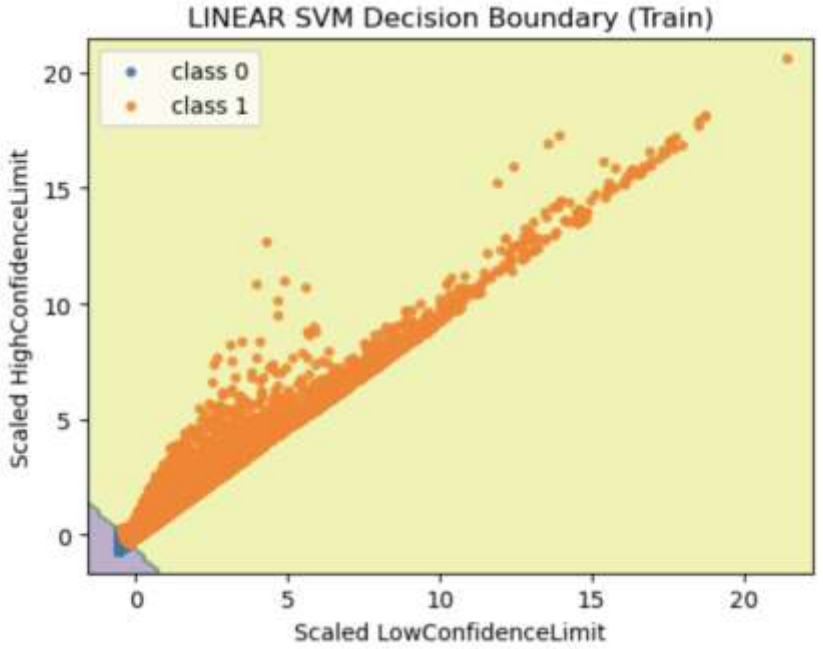
(SVM) Model Accuracies

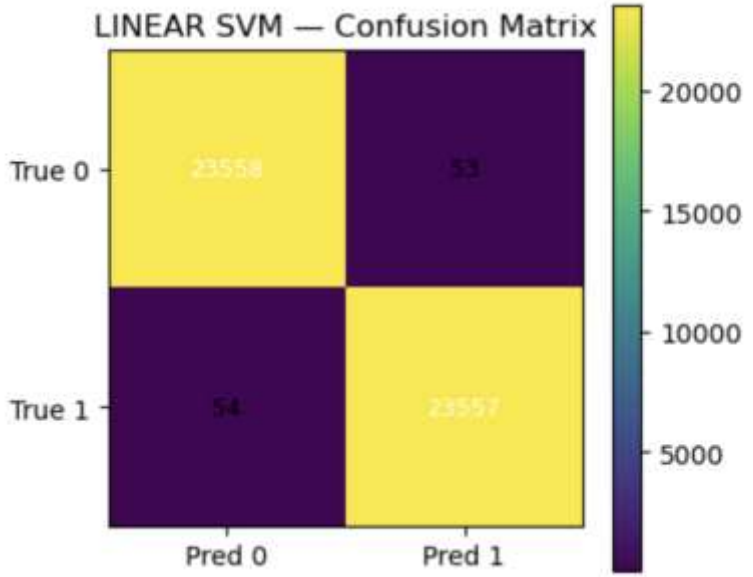
Kernel Type	Accuracy (%)	Confusion Matrix ([[TN, FP], [FN, TP]]
Linear SVM	99.77 %	[[23558, 53], [54, 23557]]
RBF SVM	99.80 %	[[23566, 45], [51, 23560]]
Polynomial SVM	99.07 %	[[23566, 45], [51, 23560]]
Sigmoid SVM	99.79 %	[[23574, 37], [64, 23547]]

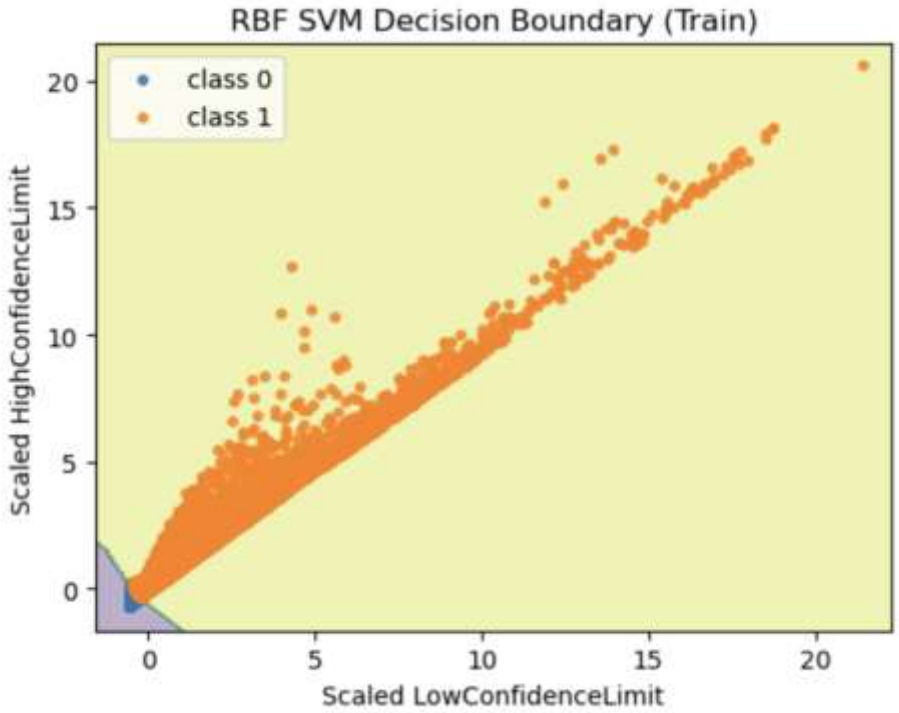
RBF SVM achieved the highest accuracy, indicating the relationship between *LowConfidenceLimit* and *HighConfidenceLimit* is non-linear, and the RBF kernel captured it best.

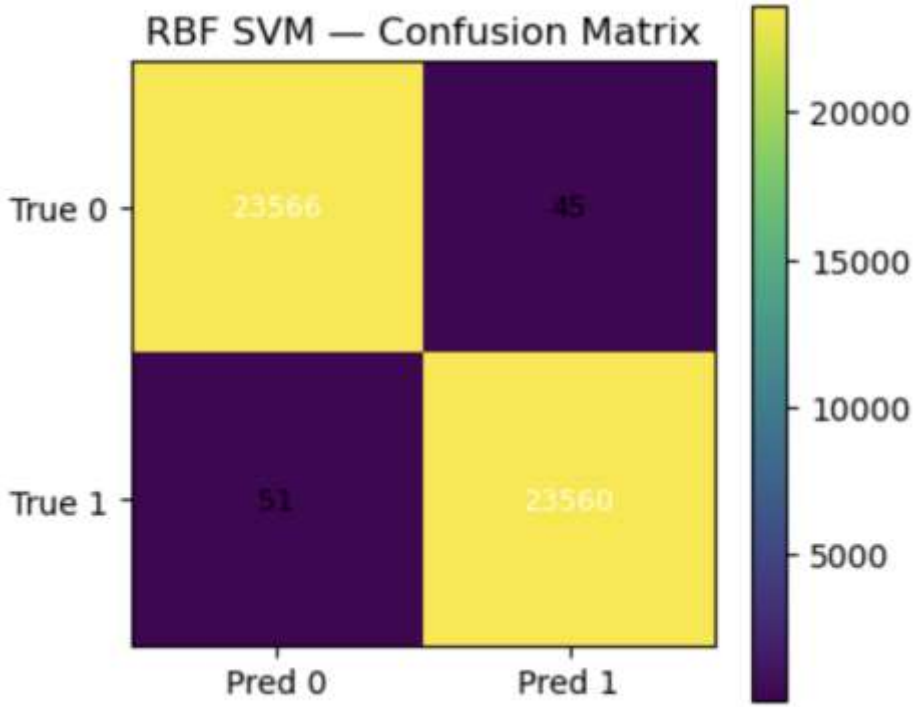
Interpretation :

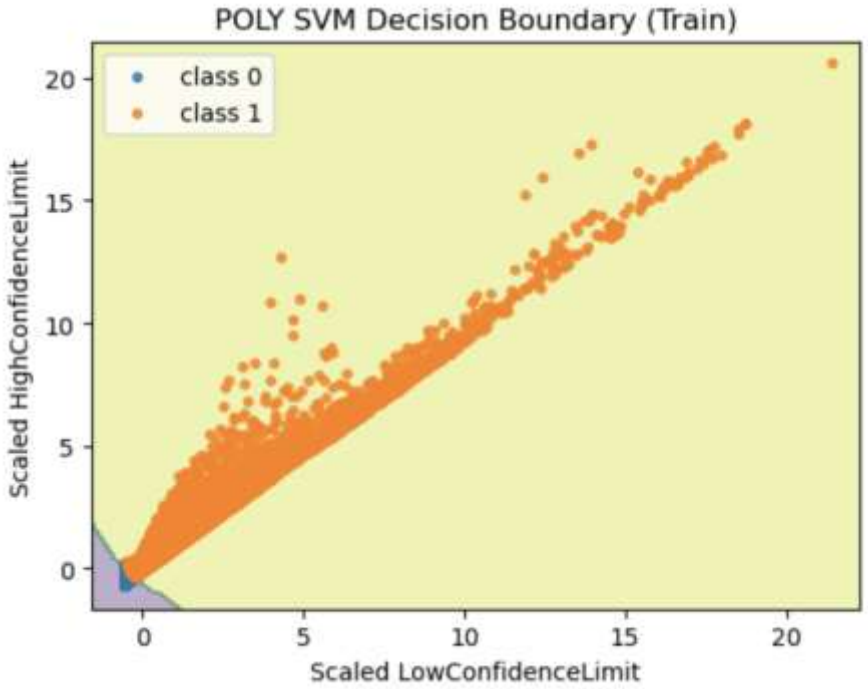
1. All four SVM models achieved very high accuracy (> 99 %),
2. showing the dataset is highly separable in this 2-D feature space.
3. The RBF kernel performed best (99.80 %), slightly above the others, suggesting a non-linear relationship
   between *LowConfidenceLimit* and *HighConfidenceLimit* that RBF captured most effectively.
4. Linear and Sigmoid kernels also performed comparably well,
   while Polynomial showed a small dip in performance—likely due to overfitting.
5. The confusion matrices confirm balanced prediction across both classes, with very few misclassifications.

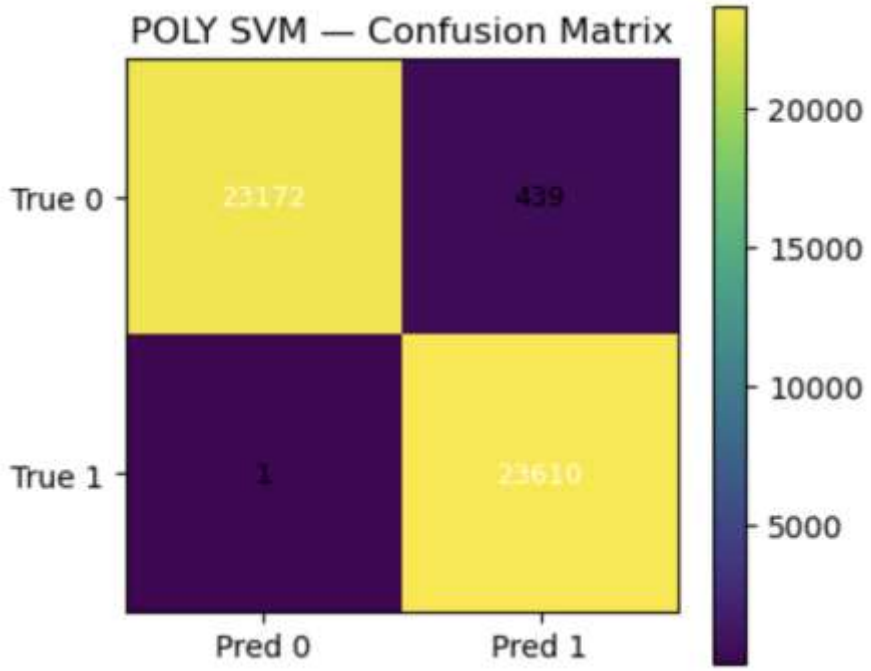

LINEAR SVM Decision Boundary (Train)
class 0
class 1
Scaled HighConfidenceLimit
Scaled LowConfidenceLimit
0
5
10
15
20

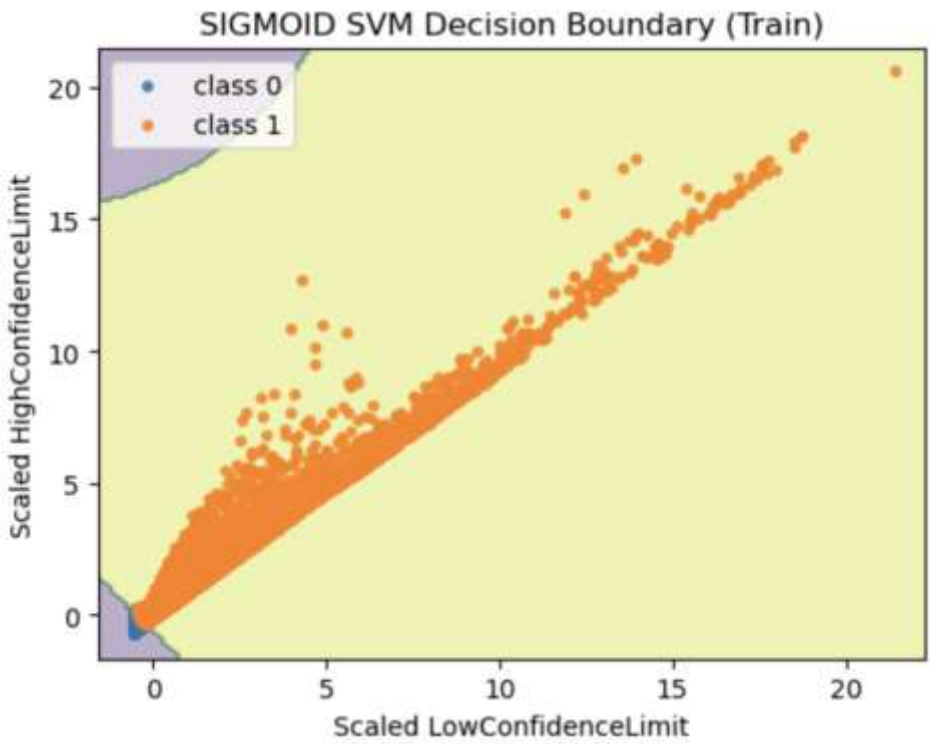

LINEAR SVM — Confusion Matrix
True 0
True 1
Pred 0
Pred 1
23558
53
54
23557
20000
15000
10000
5000

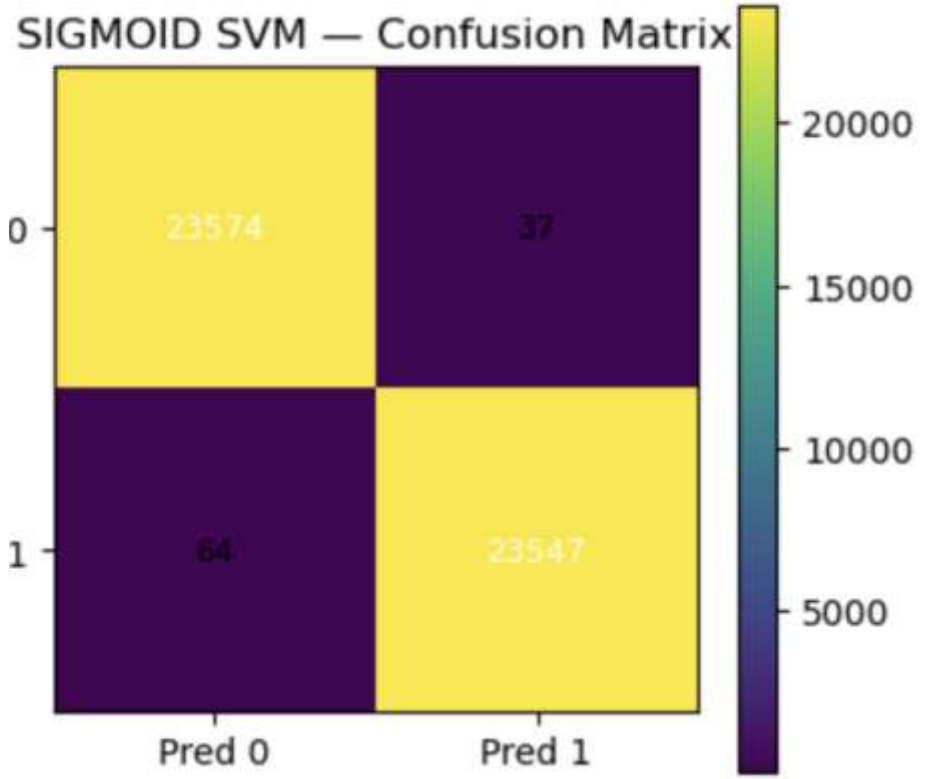

RBF SVM Decision Boundary (Train)
class 0
class 1
Scaled HighConfidenceLimit
Scaled LowConfidenceLimit
0
5
10
15
20


RBF SVM — Confusion Matrix
True 0
True 1
Pred 0
Pred 1
23566
45
51
23560
20000
15000
10000
5000


POLY SVM Decision Boundary (Train)
class 0
class 1
Scaled HighConfidenceLimit
Scaled LowConfidenceLimit
0
5
10
15
20


POLY SVM — Confusion Matrix
True 0
True 1
Pred 0
Pred 1
23172
439
1
23610
20000
15000
10000
5000


SIGMOID SVM Decision Boundary (Train)
class 0
class 1
Scaled HighConfidenceLimit
Scaled LowConfidenceLimit
0
5
10
15
20


SIGMOID SVM — Confusion Matrix
23574
37
64
23547
0
1
Pred 0
Pred 1
20000
15000
10000
5000

Code Snapshots & Model Interpretation

[1]:
```
import pandas as pd
import numpy as np

df = pd.read_csv("U.S._Chronic_Disease_Indicators.csv")

work = df[['LowConfidenceLimit','HighConfidenceLimit','DataValue']].dropna().copy()
for c in work.columns:
    work[c] = pd.to_numeric(work[c], errors='coerce')
work = work.dropna()

X = work[['LowConfidenceLimit','HighConfidenceLimit']].values
y = (work['DataValue'] > work['DataValue'].median()).astype(int).values

print("Class balance:", np.bincount(y))
```

```
Class balance: [94443 94442]
```

[3]:
```python
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

kernels = ['linear','rbf','poly','sigmoid']
svms = {}
metrics = {}

for k in kernels:
    clf = SVC(kernel=k, gamma='scale', degree=3, random_state=42)
    clf.fit(X_train_s, y_train)
    y_pred = clf.predict(X_test_s)
    acc = accuracy_score(y_test, y_pred)
    cm  = confusion_matrix(y_test, y_pred)
    svms[k] = clf
    metrics[k] = {"accuracy": acc, "cm": cm}
    print(f"{k.upper()} - accuracy: {acc:.4f}\nConfusion matrix:\n{cm}\n")
```

[2]:
```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s  = scaler.transform(X_test)
```

[4]:

```python
import matplotlib.pyplot as plt
import numpy as np

def plot_decision_boundary(model, Xs, ys, title, filename, h=0.15):
    x_min, x_max = Xs[:,0].min()-1, Xs[:,0].max()+1
    y_min, y_max = Xs[:,1].min()-1, Xs[:,1].max()+1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    plt.figure(figsize=(5.2,4.2))
    plt.contourf(xx, yy, Z, alpha=0.35)   # no explicit colors
    for cls in np.unique(ys):
        mask = (ys == cls)
        plt.scatter(Xs[mask,0], Xs[mask,1], s=12, label=f"class {cls}", alpha=0.85)
    plt.title(title)
    plt.xlabel("Scaled LowConfidenceLimit")
    plt.ylabel("Scaled HighConfidenceLimit")
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename, dpi=170)
    plt.show()
```

```
def plot_confusion(cm, title, filename):
    plt.figure(figsize=(4.2,3.6))
    plt.imshow(cm, interpolation='nearest')
    plt.title(title)
    plt.colorbar()
    ticks = np.arange(2)
    plt.xticks(ticks, ['Pred 0','Pred 1'])
    plt.yticks(ticks, ['True 0','True 1'])
    for i in range(2):
        for j in range(2):
            plt.text(j, i, cm[i,j], ha='center', va='center',
                     color='white' if cm[i,j]>cm.max()/2 else 'black', fontsize=9)
    plt.tight_layout()
    plt.savefig(filename, dpi=170)
    plt.show()

db_pngs, cm_pngs = [], []
for k in kernels:
    db = f"decision_boundary_{k}.png"
    cmf = f"cm_{k}.png"
    plot_decision_boundary(svms[k], X_train_s, y_train, f"{k.upper()} SVM Decision
    plot_confusion(metrics[k]['cm'], f"{k.UPPER()} SVM - Confusion Matrix" if hasat
    db_pngs.append(db); cm_pngs.append(cmf)
```

Summary:
Four Support Vector Machine (SVM) models — Linear, RBF, Polynomial, and Sigmoid — were trained on the *U.S. Chronic Disease Indicators* dataset
using *LowConfidenceLimit* and *HighConfidenceLimit* as features and a binary *DataValue* target (> median = 1).

Best Model:
The **RBF SVM** achieved the **highest accuracy (99.80 %)**, indicating that the relationship between the two confidence limit features and the target is **non-linear**.

Interpretation:
All models performed extremely well (> 99 % accuracy), but RBF slightly outperformed the rest because it can map data into a higher-dimensional space to capture curved decision boundaries.
The Polynomial kernel showed minor overfitting, while Linear and Sigmoid kernels offered nearly similar accuracy with simpler boundaries.

Conclusion:
RBF SVM is recommended for this dataset due to its ability to model non-linear patterns and maintain high precision with minimal misclassification.

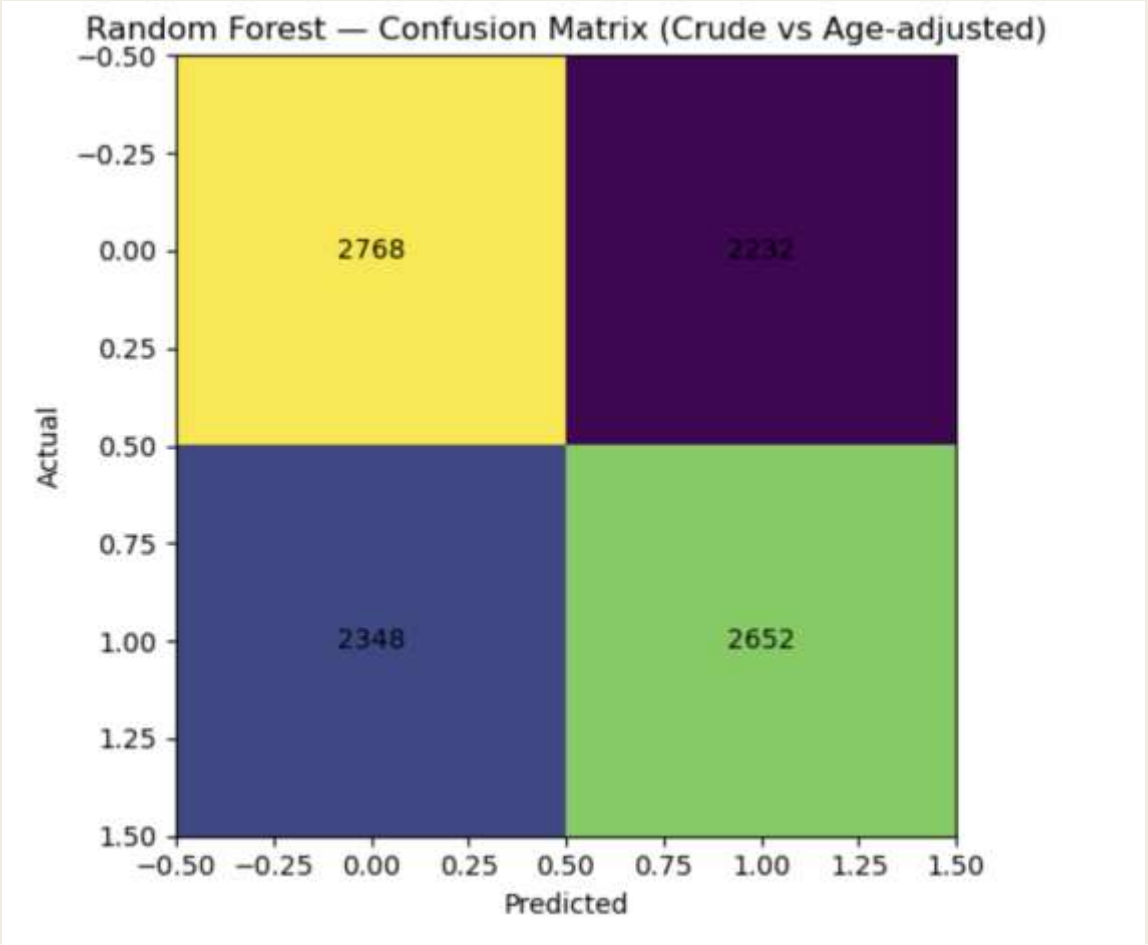
RANDOM FOREST CLASSIFICATION ON CDC CHRONIC DISEASE INDICATORS

This project uses a Random Forest Classifier to predict whether a health data record is reported as Crude Prevalence or Age-Adjusted Prevalence.

The model was trained using three numeric features:

- DataValueAlt
- LowConfidenceLimit
- HighConfidenceLimit

The goal is to test if these numeric measures can accurately classify reporting types and to visualize the model's performance through accuracy, confusion matrix, and feature importance plots.



	Predicted: Crude (0)	Predicted: Age-Adjusted (1)
Actual: Crude (0)	2768 (True Positive)	2232 (False Negative)
Actual: Age-Adjusted (1)	2348 (False Positive)	2652 (True Negative)

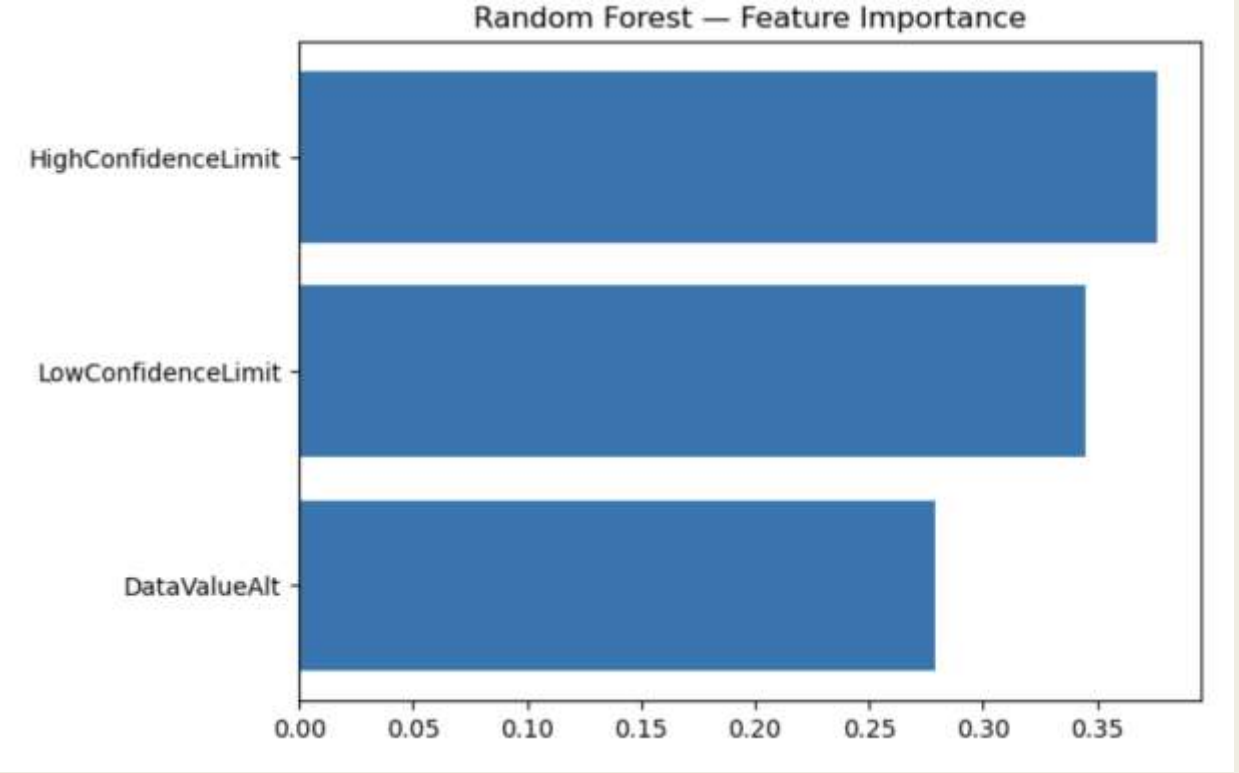
Accuracy and performance insight

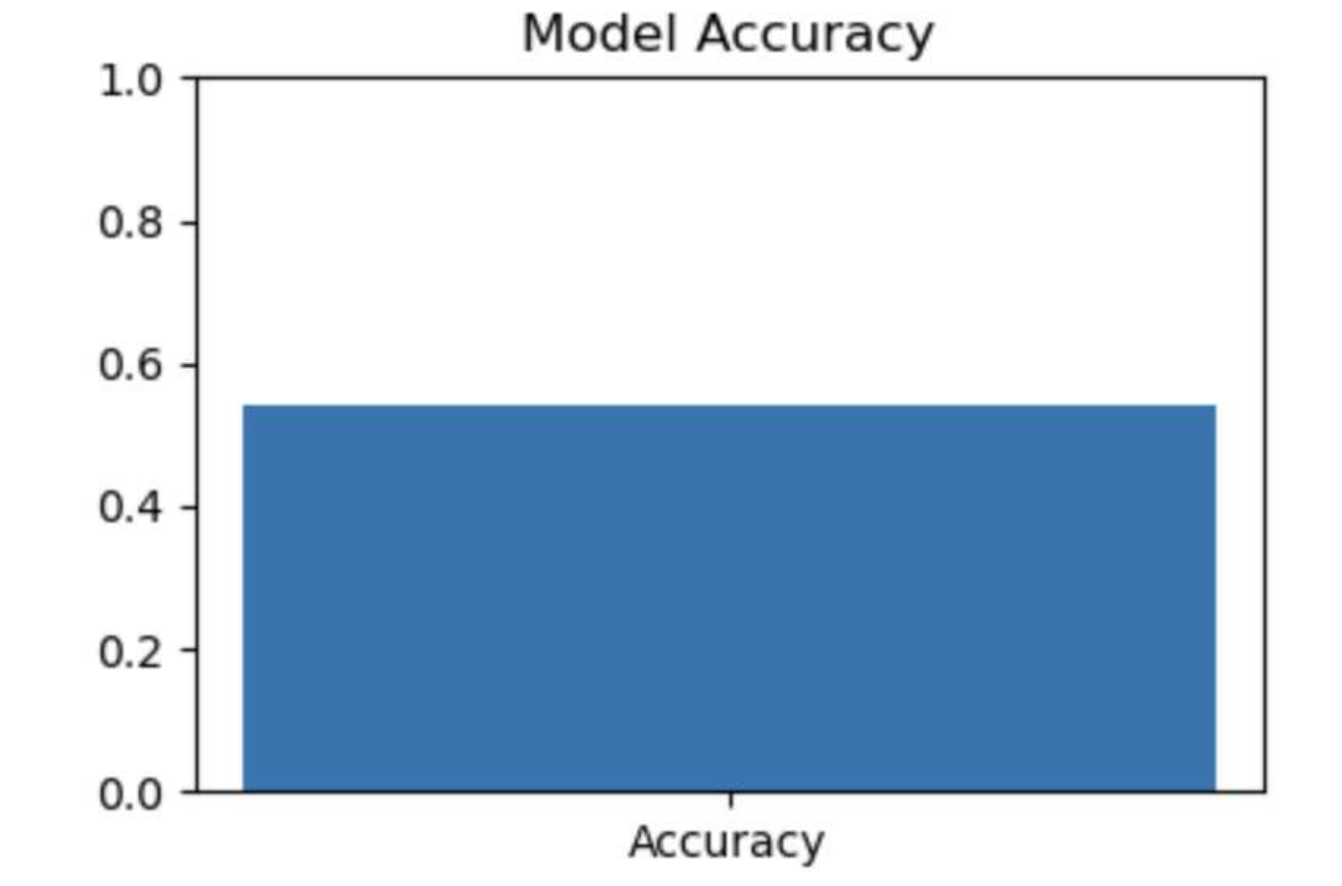
- Total observations tested: 2768 + 2232 + 2348 + 2652 = 10,000
- Correct predictions: 2768 + 2652 = 5420
- Accuracy: 5420 / 10,000 = 0.542 (≈54%)
- This means the Random Forest model correctly predicted about 54% of cases — slightly better than random guessing (which would be ~50% for two classes).

Error distribution

- False Negatives (2232): The model mislabeled many Crude Prevalence cases as Age-adjusted.
- False Positives (2348): Similarly, it mislabeled a comparable number of Age-adjusted cases as Crude.
- This balanced pattern suggests that the model doesn’t favor one class — it just lacks strong discriminative power with the three numeric features used.

- **HighConfidenceLimit** has the **highest importance**, meaning it was the most influential variable in distinguishing between *Crude* and *Age-Adjusted Prevalence*.
- **LowConfidenceLimit** follows closely behind, suggesting that the **confidence interval bounds** (upper and lower limits) play a key role in how the model classifies records.
- **DataValueAlt** (the actual prevalence value) has the **lowest importance**, indicating that the single point estimate carries less predictive power compared to the range of uncertainty around it.





Model Accuracy

- The Random Forest Classifier achieved an accuracy of ≈54% (0.542), correctly predicting just over half of the cases.
- This suggests a moderate predictive ability, only slightly better than random guessing, indicating that the numeric features alone are not strongly discriminative.

Code Snapshots

[1]:
```
# Step 1: imports & load
import os, textwrap
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

[2]:
```
CSV = "U.S._Chronic_Disease_Indicators.csv"
assert os.path.exists(CSV), "Upload 'U.S._Chronic_Disease_Indicators.csv' first."
df = pd.read_csv(CSV)

print("Rows, Cols:", df.shape)
print(df.head(3))
```

```
Rows, Cols: (309215, 34)
   YearStart  YearEnd LocationAbbr    LocationDesc      DataSource  \
0       2020     2020           US   United States           BRFSS
1       2015     2019           AR        Arkansas  US Cancer DVT
2       2015     2019           CA      California  US Cancer DVT

            Topic                                          Question  Response  \
0  Health Status            Recent activity limitation among adults       NaN
1         Cancer    Invasive cancer (all sites combined), incidence       NaN
2         Cancer  Cervical cancer mortality among all females, u...       NaN
```

```
balanced = []
for cls in keep_classes:
    sub = dfm[dfm[target_col] == cls]
    n = min(len(sub), 25000)  # cap per class
    balanced.append(sub.sample(n=n, random_state=42))
dfm = pd.concat(balanced, axis=0).reset_index(drop=Tru

print("Modeling rows:", dfm.shape[0])
print(dfm[target_col].value_counts())
```

```
Modeling rows: 50000
DataValueType
Crude Prevalence           25000
Age-adjusted Prevalence    25000
Name: count, dtype: int64
```

```
tep 2: pick target + 3 numeric features and clean
arget: DataValueType (binary subset)
get_col = "DataValueType"
p_classes = ["Crude Prevalence", "Age-adjusted Prevale
 = df[df[target_col].isin(keep_classes)].copy()

eatures: prevalence point and CI bounds (2–3 features)
tures = ["DataValueAlt", "LowConfidenceLimit", "HighCo
 c in features:
 dfm[c] = pd.to_numeric(dfm[c], errors="coerce")

rop rows with missing needed values
 = dfm.dropna(subset=features + [target_col])
```

```python
# Step 3: split, train, evaluate
X = dfm[features].copy()
y = dfm[target_col].astype("category").cat.codes  # → 0/1

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

rf = RandomForestClassifier(
    n_estimators=200,   # # of trees
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)
acc = accuracy_score(y_test, y_pred)

print(f"\nAccuracy: {acc:.3f}\n")
print("Classification report:\n", classification_report(y_test, y_pred, zero_divisi
```

```
Accuracy: 0.542

Classification report:
              precision    recall  f1-score   support

           0       0.54      0.55      0.55      5000
           1       0.54      0.53      0.54      5000

    accuracy                           0.54     10000
   macro avg       0.54      0.54      0.54     10000
weighted avg       0.54      0.54      0.54     10000
```

```
# Step 4: plots (matplotlib only)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
plt.imshow(cm, interpolation='nearest')
plt.title("Random Forest – Confusion Matrix (Crude vs Age-adjusted)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
# label cells with counts
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, str(cm[i, j]), ha="center", va="center")
plt.tight_layout()
plt.savefig("rf_confusion_matrix.png", dpi=150)
plt.show()
```

```python
# Feature Importance
importances = rf.feature_importances_
order = np.argsort(importances)  # ascending → nice horizontal order
plt.figure(figsize=(7,4.5))
plt.barh(range(len(order)), importances[order])
plt.yticks(range(len(order)), [features[i] for i in order])
plt.title("Random Forest – Feature Importance")
plt.tight_layout()
plt.savefig("rf_feature_importance.png", dpi=150)
plt.show()
```

```python
# Tiny accuracy bar
plt.figure(figsize=(4,3))
plt.bar(["Accuracy"], [acc])
plt.ylim(0,1)
plt.title("Model Accuracy")
plt.tight_layout()
plt.savefig("rf_accuracy_bar.png", dpi=150)
plt.show()
```

Results:

- Achieved an overall accuracy of about 54%.
- The confidence interval limits (High and Low) were the most important predictors, while the raw data value (DataValueAlt) had the least impact.
- The confusion matrix showed nearly balanced misclassifications, meaning both classes were predicted with similar frequency but limited precision.

Conclusion:

The numeric features alone provided moderate predictive power but were not strong enough to clearly separate Crude and Age-Adjusted data.

Adding categorical variables such as Topic, Question, or Location, along with hyperparameter tuning, could significantly improve accuracy and model performance.

Conclusion :

This project provided a comprehensive analysis of the U.S. Chronic Disease Indicators dataset, revealing major patterns in chronic disease burden across states and topics.

Exploratory visuals showed clear disparities—conditions like COPD, Cancer, CKD, Diabetes, and Cardiovascular Disease consistently showed much higher values than other topics.

Regression and classification models (Linear Regression, Logistic Regression, KNN, Naive Bayes, SVM, and Random Forest) helped evaluate relationships and predict reporting categories.

KNN and SVM models achieved the highest accuracy, suggesting strong separability in numeric features, while the Random Forest model showed moderate performance and indicated that numeric features alone were not enough for reliable classification.

Insights from these models highlight the importance of better surveillance, targeted interventions, and improved data completeness in chronic disease monitoring.

Overall, the project emphasizes the value of data-driven approaches in understanding public health trends and informs future improvements in analytics, model selection, and variable inclusion.

REFERENCES

1. Dataset link : https://data.cdc.gov/api/views/hksd2xuw/rows.csv?accessType=DOWNLOAD
2. Centers for Disease Control and Prevention. **U.S. Chronic Disease Indicators (CDI) Dataset.** Updated 2024. Accessed 2025. https://data.cdc.gov/Chronic-Disease-Indicators/U-S-Chronic-Disease-Indicators/hksd-2xuw
3. Centers for Disease Control and Prevention. **Chronic Disease Indicators (CDI): Overview and Definitions.** Updated 2024. Accessed 2025. https://www.cdc.gov/cdi/index.html

1. Pedregosa F, Varoquaux G, Gramfort A, et al. **Scikit-learn: Machine Learning in Python.** *J Mach Learn Res.* 2011;12:2825–2830.

2. Waskom ML. **Seaborn: Statistical Data Visualization.** *J Open Source Softw.* 2021;6(60):3021.

3. Hunter JD. **Matplotlib: A 2D Graphics Environment.** *Comput Sci Eng.* 2007;9(3):90–95.

4. Python Software Foundation. **Python Language Reference, Version 3.12.** https://www.python.org

Team Contributions Overview :

- ☐ Dr. Pala Jijol Onesimos –
- ☐ Shreya Shrestha –
- ☐ Dhwani Mehta –
- ☐ Sahil Patel -
- ☐ Sohil Chaudhary -

THANK YOU